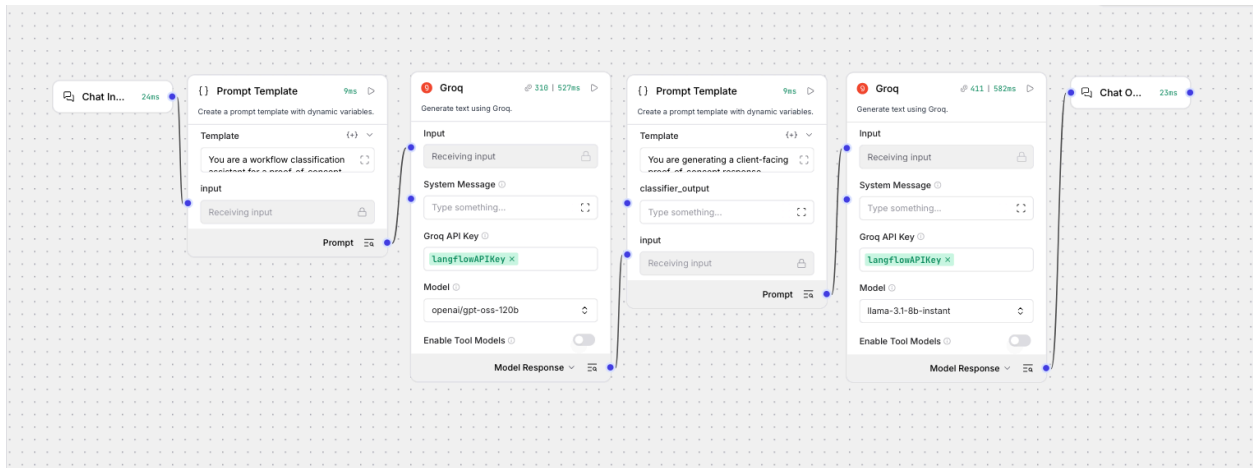


Langflow created as below

Used flow: Chat Input → Prompt Template → LLM → Prompt Template → LLM → Chat Output



Prompt template 1:

Prompt used

You are a workflow classification assistant for a proof-of-concept demo.

Your task is to classify the user's request into exactly one supported category.

Supported categories:

1. bank-collector_settled
2. bank-transaction_rejected
3. bank-transaction_settled
4. bank-details_found
5. bank-details_not-found
6. bank-details_requested
7. CREDIT-CARD-STL-FEES_RECEIVED
8. CREDIT-CARD-STL-ITEM_RECEIVED
9. CREDIT-CARD-STL-PAYOUT_RECEIVED

User request:

{input}

LLM 1 setting:

Temperature 0.1

Model: openai/gpt-oss-120b

Prompt template 2:

Prompt used

You are generating a client-facing proof-of-concept response.

Original user request:

{input}

Classifier result:

{classifier_output}

Generate a polished response with the following sections:

1. Request Summary
2. Selected Template Category
3. Demo Response
4. Sample Structured Output
5. Business Value

Rules:

- Do not mention internal code, hidden logic, browser extension details, or proprietary implementation.
- Keep the tone professional and presentation-friendly.
- The Sample Structured Output must be valid JSON.
- Keep the response concise but clear.

LLM 2 setting:

Temperature 0.3

Model: openai.gpt-oss-120b

User Input : Generate a settled bank transaction communication



User

Generate a settled bank transaction communication

Response from AI

****Bank Transaction Settled Proof-of-Concept Response****

Request Summary

The client has requested a proof-of-concept response for the "bank-transaction_settled" event. This response aims to demonstrate a clear and concise representation of the transaction settlement status.

Selected Template Category

Based on the request, we have selected a template category that focuses on financial transactions and settlement events.

Demo Response

****Transaction Settled Response****

The transaction has been successfully settled.

Sample Structured Output

```
```json
{
 "transaction_id": "TXN-12345",
 "settlement_status": "settled",
 "settlement_date": "2023-02-20T14:30:00.000Z",
 "amount": 100.00,
 "currency": "USD"
}
```
```

Business Value

This proof-of-concept response provides a clear and standardized representation of the transaction settlement status, enabling seamless integration with various systems and applications. The structured output format facilitates easy data consumption and processing, ultimately enhancing the overall efficiency and accuracy of financial transactions.



✓ Finished in

721 | 1.2s

Bank Transaction Settled Proof-of-Concept Response

Request Summary

The client has requested a proof-of-concept response for the "bank-transaction_settled" event. This response aims to demonstrate a clear and concise representation of the transaction settlement status.

Selected Template Category

Based on the request, we have selected a template category that focuses on financial transactions and settlement events.

Demo Response

Transaction Settled Response

The transaction has been successfully settled.

Sample Structured Output

json



```
{
  "transaction_id": "TXN-12345",
  "settlement_status": "settled",
  "settlement_date": "2023-02-20T14:30:00.000Z",
  "amount": 100.00,
  "currency": "USD"
}
```

Business Value

This proof-of-concept response provides a clear and standardized representation of the transaction settlement status, enabling seamless integration with various systems and applications. The structured output format facilitates easy data consumption and processing, ultimately enhancing the overall efficiency and accuracy of financial transactions.

Exported json flow content:

```
{
  "data": {
    "edges": [
      {
        "animated": false,
        "className": "",
        "data": {
          "sourceHandle": {
            "dataType": "ChatInput",
            "id": "ChatInput-fXqWc",
            "name": "message",
            "output_types": [
              "Message"
            ]
          },
          "targetHandle": {
            "fieldName": "input",
            "id": "Prompt Template-AVLhB",
            "inputTypes": [
              "Message"
            ],
            "type": "str"
          }
        },
        "id":
"xy-edge__ChatInput-fXqWc{œdataTypeœ:œChatInputœ,œidœ:œChatInput-fXqWcœ,œnameœ:œ
messageœ,œoutput_typesœ:[œMessageœ]}-Prompt
Template-AVLhB{œfieldNameœ:œinputœ,œidœ:œPrompt
Template-AVLhBœ,œinputTypesœ:[œMessageœ],œtypeœ:œstrœ}",
        "selected": false,
        "source": "ChatInput-fXqWc",
        "sourceHandle":
"{œdataTypeœ:œChatInputœ,œidœ:œChatInput-fXqWcœ,œnameœ:œmessageœ,œoutput_types
œ:[œMessageœ]}",
        "target": "Prompt Template-AVLhB",
        "targetHandle": "{œfieldNameœ:œinputœ,œidœ:œPrompt
Template-AVLhBœ,œinputTypesœ:[œMessageœ],œtypeœ:œstrœ}"
      },
      {
        "animated": false,
        "className": "",
        "data": {
          "sourceHandle": {
            "dataType": "Prompt Template",
```

```

    "id": "Prompt Template-AVLhB",
    "name": "prompt",
    "output_types": [
        "Message"
    ]
},
"targetHandle": {
    "fieldName": "input_value",
    "id": "GroqModel-AjfvL",
    "inputTypes": [
        "Message"
    ],
    "type": "str"
}
},
"id": "xy-edge__Prompt Template-AVLhB{ædataTypeæ:æPrompt Templateæ,æidæ:æPrompt
Template-AVLhBæ,ænameæ:æpromptæ,æoutput_typesæ:[æMessageæ]}-GroqModel-AjfvL{æfield
Nameæ:æinput_valueæ,æidæ:æGroqModel-AjfvLæ,æinputTypesæ:[æMessageæ],ætypeæ:æstr
æ}",
    "selected": false,
    "source": "Prompt Template-AVLhB",
    "sourceHandle": "{ædataTypeæ:æPrompt Templateæ,æidæ:æPrompt
Template-AVLhBæ,ænameæ:æpromptæ,æoutput_typesæ:[æMessageæ]}",
    "target": "GroqModel-AjfvL",
    "targetHandle":
"{æfieldNameæ:æinput_valueæ,æidæ:æGroqModel-AjfvLæ,æinputTypesæ:[æMessageæ],ætype
æ:æstræ}"
},
{
    "animated": false,
    "className": "",
    "data": {
        "sourceHandle": {
            "dataType": "GroqModel",
            "id": "GroqModel-AjfvL",
            "name": "text_output",
            "output_types": [
                "Message"
            ]
        },
        "targetHandle": {
            "fieldName": "input",
            "id": "Prompt Template-pxOTt",
            "inputTypes": [
                "Message"
            ],

```

```

    "type": "str"
  }
},
"id":
"xy-edge__GroqModel-AjfvL{œdataTypeœ:œGroqModelœ,œidœ:œGroqModel-AjfvLœ,œnameœ:œ
text_outputœ,œoutput_typesœ:[œMessageœ]}-Prompt
Template-pxOTt{œfieldNameœ:œinputœ,œidœ:œPrompt
Template-pxOTtœ,œinputTypesœ:[œMessageœ],œtypeœ:œstrœ}",
  "selected": false,
  "source": "GroqModel-AjfvL",
  "sourceHandle":
"{œdataTypeœ:œGroqModelœ,œidœ:œGroqModel-AjfvLœ,œnameœ:œtext_outputœ,œoutput_ty
pesœ:[œMessageœ]}",
  "target": "Prompt Template-pxOTt",
  "targetHandle": "{œfieldNameœ:œinputœ,œidœ:œPrompt
Template-pxOTtœ,œinputTypesœ:[œMessageœ],œtypeœ:œstrœ}"
},
{
  "animated": false,
  "className": "",
  "data": {
    "sourceHandle": {
      "dataType": "Prompt Template",
      "id": "Prompt Template-pxOTt",
      "name": "prompt",
      "output_types": [
        "Message"
      ]
    },
    "targetHandle": {
      "fieldName": "input_value",
      "id": "GroqModel-Zbm2F",
      "inputTypes": [
        "Message"
      ],
      "type": "str"
    }
  },
  "id": "xy-edge__Prompt Template-pxOTt{œdataTypeœ:œPrompt Templateœ,œidœ:œPrompt
Template-pxOTtœ,œnameœ:œpromptœ,œoutput_typesœ:[œMessageœ]}-GroqModel-Zbm2F{œfiel
dNameœ:œinput_valueœ,œidœ:œGroqModel-Zbm2Fœ,œinputTypesœ:[œMessageœ],œtypeœ:œs
trœ}",
  "selected": false,
  "source": "Prompt Template-pxOTt",
  "sourceHandle": "{œdataTypeœ:œPrompt Templateœ,œidœ:œPrompt
Template-pxOTtœ,œnameœ:œpromptœ,œoutput_typesœ:[œMessageœ]}",

```

```

        "target": "GroqModel-Zbm2F",
        "targetHandle":
        "{æfieldNameæ:æinput_valueæ,æidæ:æGroqModel-Zbm2Fæ,æinputTypesæ:[æMessageæ],ætyp
eæ:æstroæ}"
    },
    {
        "animated": false,
        "className": "",
        "data": {
            "sourceHandle": {
                "dataType": "GroqModel",
                "id": "GroqModel-Zbm2F",
                "name": "text_output",
                "output_types": [
                    "Message"
                ]
            },
            "targetHandle": {
                "fieldName": "input_value",
                "id": "ChatOutput-D1HOI",
                "inputTypes": [
                    "Data",
                    "JSON",
                    "DataFrame",
                    "Table",
                    "Message"
                ],
                "type": "other"
            }
        },
        "id":
        "xy-edge__GroqModel-Zbm2F{ædataTypeæ:æGroqModelæ,æidæ:æGroqModel-Zbm2Fæ,æname
æ:ætext_outputæ,æoutput_typesæ:[æMessageæ]}-ChatOutput-D1HOI{æfieldNameæ:æinput_valu
eæ,æidæ:æChatOutput-D1HOIæ,æinputTypesæ:[æDataæ,æJSONæ,æDataFrameæ,æTableæ,æ
Messageæ],ætypeæ:æotheræ}",
        "selected": false,
        "source": "GroqModel-Zbm2F",
        "sourceHandle":
        "{ædataTypeæ:æGroqModelæ,æidæ:æGroqModel-Zbm2Fæ,ænameæ:ætext_outputæ,æoutput_ty
pesæ:[æMessageæ]}",
        "target": "ChatOutput-D1HOI",
        "targetHandle":
        "{æfieldNameæ:æinput_valueæ,æidæ:æChatOutput-D1HOIæ,æinputTypesæ:[æDataæ,æJSONæ
æ,æDataFrameæ,æTableæ,æMessageæ],ætypeæ:æotheræ}"
    }
],

```



```

"nodes": [
  {
    "data": {
      "id": "ChatInput-fXqWc",
      "node": {
        "base_classes": [
          "Message"
        ],
        "beta": false,
        "conditional_paths": [],
        "custom_fields": {},
        "description": "Get chat inputs from the Playground.",
        "display_name": "Chat Input",
        "documentation": "https://docs.langflow.org/chat-input-and-output",
        "edited": false,
        "field_order": [
          "input_value",
          "should_store_message",
          "sender",
          "sender_name",
          "session_id",
          "context_id",
          "files"
        ],
        "frozen": false,
        "icon": "MessagesSquare",
        "legacy": false,
        "lf_version": "1.9.1",
        "metadata": {
          "code_hash": "7a26c54d89ed",
          "dependencies": {
            "dependencies": [
              {
                "name": "lfx",
                "version": null
              }
            ]
          },
          "total_dependencies": 1
        },
        "module": "lfx.components.input_output.chat.ChatInput"
      },
      "minimized": true,
      "output_types": [],
      "outputs": [
        {
          "allows_loop": false,

```

```

    "cache": true,
    "display_name": "Chat Message",
    "group_outputs": false,
    "method": "message_response",
    "name": "message",
    "selected": "Message",
    "tool_mode": true,
    "types": [
      "Message"
    ],
    "value": "__UNDEFINED__"
  },
  {
    "pinned": false,
    "template": {
      "_type": "Component",
      "code": {
        "advanced": true,
        "dynamic": true,
        "fileTypes": [],
        "file_path": "",
        "info": "",
        "list": false,
        "load_from_db": false,
        "multiline": true,
        "name": "code",
        "password": false,
        "placeholder": "",
        "required": true,
        "show": true,
        "title_case": false,
        "type": "code",
        "value": "from lfx.base.data.utils import IMG_FILE_TYPES, TEXT_FILE_TYPES\nfrom\nlfx.base.io.chat import ChatComponent\nfrom lfx.inputs.inputs import BoolInput\nfrom lfx.io import (\nDropdownInput,\n FileInput,\n MessageTextInput,\n MultilineInput,\n Output,\n)\nfrom\nlfx.schema.message import Message\nfrom lfx.utils.constants import (\nMESSAGE_SENDER_AI,\n MESSAGE_SENDER_NAME_USER,\nMESSAGE_SENDER_USER,\n)\n\n\nclass ChatInput(ChatComponent):\n    display_name = \"Chat\nInput\"\n    description = \"Get chat inputs from the Playground.\"\n    documentation: str =\n\"https://docs.langflow.org/chat-input-and-output\"\n    icon = \"MessagesSquare\"\n    name =\n\"ChatInput\"\n    minimized = True\n    inputs = [\n        MultilineInput(\n            name=\"input_value\",\n            display_name=\"Input Text\",\n            value=\"\", \n            info=\"Message to be passed as input.\", \n            input_types=[], \n        ), \n        BoolInput(\n            name=\"should_store_message\",\n            display_name=\"Store Messages\", \n            info=\"Store\nthe message in the history.\", \n            value=True, \n            advanced=True, \n        ), \n        DropdownInput(\n            name=\"sender\", \n            display_name=\"Sender Type\", \n
```

```

options=[MESSAGE_SENDER_AI, MESSAGE_SENDER_USER],\n
value=MESSAGE_SENDER_USER,\n      info="Type of sender.",\n      advanced=True,\n
),\n      MessageTextInput(\n      name="sender_name",\n      display_name="Sender\nName",\n      info="Name of the sender.",\n
value=MESSAGE_SENDER_NAME_USER,\n      advanced=True,\n      ),\n
MessageTextInput(\n      name="session_id",\n      display_name="Session ID",\n
info="The session ID of the chat. If empty, the current session ID parameter will be used.",\n
advanced=True,\n      ),\n      MessageTextInput(\n      name="context_id",\n
display_name="Context ID",\n      info="The context ID of the chat. Adds an extra layer to the\nlocal memory.",\n      value="",\n      advanced=True,\n      ),\n      FileInput(\n
name="files",\n      display_name="Files",\n      file_types=TEXT_FILE_TYPES +\nIMG_FILE_TYPES,\n      info="Files to be sent with the message.",\n      advanced=True,\n
is_list=True,\n      temp_file=True,\n      ),\n      ]\n      outputs = [\n
Output(display_name="Chat Message", name="message", method="message_response"),\n
]\n\n      async def message_response(self) -> Message:\n      # Ensure files is a list and filter out\nempty/None values\n      files = self.files if self.files else []\n      if files and not isinstance(files,\nlist):\n      files = [files]\n      # Filter out None/empty values\n      files = [f for f in files if f is not\nNone and f != ""]\n      session_id = self.session_id or self.graph.session_id or ""\n
message = await Message.create(\n      text=self.input_value,\n      sender=self.sender,\n
sender_name=self.sender_name,\n      session_id=session_id,\n
context_id=self.context_id,\n      files=files,\n      )\n      if session_id and isinstance(message,\nMessage) and self.should_store_message:\n      stored_message = await self.send_message(\nmessage,\n      )\n      self.message.value = stored_message\n      message =\nstored_message\n\n      self.status = message\n      return message\n
},\n
"context_id": {\n
    "_input_type": "MessageTextInput",\n
    "advanced": true,\n
    "display_name": "Context ID",\n
    "dynamic": false,\n
    "info": "The context ID of the chat. Adds an extra layer to the local memory.",\n
    "input_types": [\n
        "Message"\n
    ],\n
    "list": false,\n
    "list_add_label": "Add More",\n
    "load_from_db": false,\n
    "name": "context_id",\n
    "override_skip": false,\n
    "placeholder": "",\n
    "required": false,\n
    "show": true,\n
    "title_case": false,\n
    "tool_mode": false,\n
    "trace_as_input": true,\n
    "trace_as_metadata": true,\n

```

```
"track_in_telemetry": false,
"type": "str",
"value": ""
},
"files": {
  "_input_type": "FileInput",
  "advanced": true,
  "display_name": "Files",
  "dynamic": false,
  "fileTypes": [
    "csv",
    "json",
    "pdf",
    "txt",
    "md",
    "mdx",
    "yaml",
    "yml",
    "xml",
    "html",
    "htm",
    "docx",
    "py",
    "sh",
    "sql",
    "js",
    "ts",
    "tsx",
    "jpg",
    "jpeg",
    "png",
    "bmp",
    "image"
  ],
  "file_path": "",
  "info": "Files to be sent with the message.",
  "list": true,
  "list_add_label": "Add More",
  "name": "files",
  "override_skip": false,
  "placeholder": "",
  "required": false,
  "show": true,
  "temp_file": true,
  "title_case": false,
  "tool_mode": false,
```

```

    "trace_as_metadata": true,
    "track_in_telemetry": false,
    "type": "file",
    "value": ""
  },
  "input_value": {
    "_input_type": "MultilineInput",
    "advanced": false,
    "ai_enabled": false,
    "copy_field": false,
    "display_name": "Input Text",
    "dynamic": false,
    "info": "Message to be passed as input.",
    "input_types": [],
    "list": false,
    "list_add_label": "Add More",
    "load_from_db": false,
    "multiline": true,
    "name": "input_value",
    "override_skip": false,
    "password": false,
    "placeholder": "",
    "required": false,
    "show": true,
    "title_case": false,
    "tool_mode": false,
    "trace_as_input": true,
    "trace_as_metadata": true,
    "track_in_telemetry": false,
    "type": "str",
    "value": ""
  },
  "sender": {
    "_input_type": "DropdownInput",
    "advanced": true,
    "combobox": false,
    "dialog_inputs": {},
    "display_name": "Sender Type",
    "dynamic": false,
    "external_options": {},
    "info": "Type of sender.",
    "name": "sender",
    "options": [
      "Machine",
      "User"
    ],
  },

```

```

"options_metadata": [],
"override_skip": false,
"placeholder": "",
"required": false,
"show": true,
"title_case": false,
"toggle": false,
"tool_mode": false,
"trace_as_metadata": true,
"track_in_telemetry": true,
"type": "str",
"value": "User"
},
"sender_name": {
  "_input_type": "MessageTextInput",
  "advanced": true,
  "display_name": "Sender Name",
  "dynamic": false,
  "info": "Name of the sender.",
  "input_types": [
    "Message"
  ],
  "list": false,
  "list_add_label": "Add More",
  "load_from_db": false,
  "name": "sender_name",
  "override_skip": false,
  "placeholder": "",
  "required": false,
  "show": true,
  "title_case": false,
  "tool_mode": false,
  "trace_as_input": true,
  "trace_as_metadata": true,
  "track_in_telemetry": false,
  "type": "str",
  "value": "User"
},
"session_id": {
  "_input_type": "MessageTextInput",
  "advanced": true,
  "display_name": "Session ID",
  "dynamic": false,
  "info": "The session ID of the chat. If empty, the current session ID parameter will be
used.",
  "input_types": [

```

```

    "Message"
  ],
  "list": false,
  "list_add_label": "Add More",
  "load_from_db": false,
  "name": "session_id",
  "override_skip": false,
  "placeholder": "",
  "required": false,
  "show": true,
  "title_case": false,
  "tool_mode": false,
  "trace_as_input": true,
  "trace_as_metadata": true,
  "track_in_telemetry": false,
  "type": "str",
  "value": ""
},
"should_store_message": {
  "_input_type": "BoolInput",
  "advanced": true,
  "display_name": "Store Messages",
  "dynamic": false,
  "info": "Store the message in the history.",
  "list": false,
  "list_add_label": "Add More",
  "name": "should_store_message",
  "override_skip": false,
  "placeholder": "",
  "required": false,
  "show": true,
  "title_case": false,
  "tool_mode": false,
  "trace_as_metadata": true,
  "track_in_telemetry": true,
  "type": "bool",
  "value": true
}
},
"tool_mode": false
},
"showNode": false,
"type": "ChatInput"
},
"dragging": false,
"id": "ChatInput-fXqWc",

```

```
"measured": {
  "height": 52,
  "width": 192
},
"position": {
  "x": 38,
  "y": 236.5
},
"selected": false,
"type": "genericNode"
},
{
  "data": {
    "id": "Prompt Template-AVLhB",
    "node": {
      "base_classes": [
        "Message"
      ],
      "beta": false,
      "conditional_paths": [],
      "custom_fields": {
        "template": [
          "input"
        ]
      },
      "description": "Create a prompt template with dynamic variables.",
      "display_name": "Prompt Template",
      "documentation": "https://docs.langflow.org/components-prompts",
      "edited": false,
      "error": null,
      "field_order": [
        "template",
        "use_double_brackets",
        "tool_placeholder"
      ],
      "frozen": false,
      "full_path": null,
      "icon": "prompts",
      "is_composition": null,
      "is_input": null,
      "is_output": null,
      "legacy": false,
      "lf_version": "1.9.1",
      "metadata": {
        "code_hash": "3d3fd7b8a36f",
        "dependencies": {
```



```

    "dependencies": [
      {
        "name": "lfx",
        "version": null
      }
    ],
    "total_dependencies": 1
  },
  "module": "lfx.components.models_and_agents.prompt.PromptComponent"
},
"minimized": false,
"name": "",
"output_types": [],
"outputs": [
  {
    "allows_loop": false,
    "cache": true,
    "display_name": "Prompt",
    "group_outputs": false,
    "hidden": null,
    "loop_types": null,
    "method": "build_prompt",
    "name": "prompt",
    "options": null,
    "required_inputs": null,
    "selected": "Message",
    "tool_mode": true,
    "types": [
      "Message"
    ],
    "value": "__UNDEFINED__"
  }
],
"pinned": false,
"priority": null,
"replacement": null,
"template": {
  "_type": "Component",
  "code": {
    "advanced": true,
    "dynamic": true,
    "fileTypes": [],
    "file_path": "",
    "info": "",
    "list": false,
    "load_from_db": false,

```

```

"multiline": true,
"name": "code",
"password": false,
"placeholder": "",
"required": true,
"show": true,
"title_case": false,
"type": "code",
"value": "from typing import Any\n\nfrom lfx.base.prompts.api_utils import
process_prompt_template\nfrom lfx.custom.custom_component.component import
Component\nfrom lfx.inputs.input_mixin import FieldTypes\nfrom lfx.inputs.inputs import
DefaultPromptField\nfrom lfx.io import BoolInput, MessageTextInput, Output, PromptInput\nfrom
lfx.log.logger import logger\nfrom lfx.schema.dotdict import dotdict\nfrom lfx.schema.message import
Message\nfrom lfx.template.utils import update_template_values\nfrom lfx.utils.mustache_security
import validate_mustache_template\n\n\nclass PromptComponent(Component):\n    display_name:
str = \"Prompt Template\"\n    description: str = \"Create a prompt template with dynamic
variables.\"\n    documentation: str = \"https://docs.langflow.org/components-prompts\"\n    icon =
\"prompts\"\n    trace_type = \"prompt\"\n    name = \"Prompt Template\"\n    inputs = [\n
PromptInput(name=\"template\", display_name=\"Template\"),\n    BoolInput(\n
name=\"use_double_brackets\", display_name=\"Use Double Brackets\", \n
value=False, \n    advanced=True, \n    info=\"Use {{variable}} syntax instead of
{variable}.\", \n    real_time_refresh=True, \n    ), \n    MessageTextInput(\n
name=\"tool_placeholder\", display_name=\"Tool Placeholder\", \n    tool_mode=True, \n
advanced=True, \n    show=False, \n    info=\"A placeholder input for tool mode.\"", \n    ), \n
]\n    outputs = [\n    Output(display_name=\"Prompt\", name=\"prompt\",
method=\"build_prompt\"), \n    ]\n\n    def update_build_config(self, build_config: dotdict, field_value:
Any, field_name: str | None = None) -> dotdict:\n        \"\"\"Update the template field type based on
the selected mode.\"\"\"\n        if field_name == \"use_double_brackets\":\n            # Change the
template field type based on mode\n            is_mustache = field_value is True\n            if
is_mustache:\n                build_config[\"template\"][\"type\"] =
FieldTypes.MUSTACHE_PROMPT.value\n            else:\n                build_config[\"template\"][\"type\"]
= FieldTypes.PROMPT.value\n            # Re-process the template to update variables when mode
changes\n            template_value = build_config.get(\"template\", {}).get(\"value\", \"\")\n            if
template_value:\n                # Ensure custom_fields is properly initialized\n                if
\"custom_fields\" not in build_config:\n                    build_config[\"custom_fields\"] = {}\n                #
Clean up fields from the OLD mode before processing with NEW mode\n                # This ensures
we don't keep fields with wrong syntax even if validation fails\n                old_custom_fields =
build_config[\"custom_fields\"].get(\"template\", [])\n                for old_field in
list(old_custom_fields):\n                    # Remove the field from custom_fields and template\n
if old_field in old_custom_fields:\n                        old_custom_fields.remove(old_field)\n
build_config.pop(old_field, None)\n                # Try to process template with new mode to add new
variables\n                # If validation fails, at least we cleaned up old fields\n                try:\n
# Validate mustache templates for security\n                    if is_mustache:\n
validate_mustache_template(template_value)\n                    # Re-process template with new mode
to add new variables\n                    _ = process_prompt_template(\n
template=template_value, \n                    name=\"template\", \n

```

```

custom_fields=build_config["custom_fields"],\n
frontend_node_template=build_config,\n                is_mustache=is_mustache,\n                )\n
except ValueError as e:\n                # If validation fails, we still updated the mode and cleaned old\n
fields\n                # User will see error when they try to save\n
logger.debug(f"Template validation failed during mode switch: {e}")\n                return build_config\n\n
async def build_prompt(self) -> Message:\n                use_double_brackets = self.use_double_brackets if\n
hasattr(self, "use_double_brackets") else False\n                template_format = "mustache" if\n
use_double_brackets else "f-string"\n                prompt = await\n
Message.from_template_and_variables(template_format=template_format, **self._attributes)\n
self.status = prompt.text\n                return prompt\n\n        def _update_template(self, frontend_node:\n
dict):\n                prompt_template = frontend_node["template"]["template"]["value"]\n
use_double_brackets = frontend_node["template"].get("use_double_brackets", {}).get("value",\n
False)\n                is_mustache = use_double_brackets is True\n\n                try:\n                # Validate mustache\n
templates for security\n                if is_mustache:\n
validate_mustache_template(prompt_template)\n\n                custom_fields =\n
frontend_node["custom_fields"]\n                frontend_node_template = frontend_node["template"]\n
_ = process_prompt_template(\n                template=prompt_template,\n
name="template",\n                custom_fields=custom_fields,\n
frontend_node_template=frontend_node_template,\n                is_mustache=is_mustache,\n
)\n                except ValueError as e:\n                # If validation fails, don't add variables but allow component\n
to be created\n                logger.debug(f"Template validation failed in _update_template: {e}")\n
return frontend_node\n\n        async def update_frontend_node(self, new_frontend_node: dict,\n
current_frontend_node: dict):\n                """This function is called after the code validation is\n
done."""\n                frontend_node = await super().update_frontend_node(new_frontend_node,\n
current_frontend_node)\n                template = frontend_node["template"]["template"]["value"]\n
use_double_brackets = frontend_node["template"].get("use_double_brackets", {}).get("value",\n
False)\n                is_mustache = use_double_brackets is True\n\n                try:\n                # Validate mustache\n
templates for security\n                if is_mustache:\n                validate_mustache_template(template)\n\n                # Kept it duplicated for backwards compatibility\n                _ = process_prompt_template(\n
template=template,\n                name="template",\n
custom_fields=frontend_node["custom_fields"],\n
frontend_node_template=frontend_node["template"],\n                is_mustache=is_mustache,\n
)\n                except ValueError as e:\n                # If validation fails, don't add variables but allow component\n
to be updated\n                logger.debug(f"Template validation failed in update_frontend_node: {e}")\n
# Now that template is updated, we need to grab any values that were set in the\n
current_frontend_node\n                # and update the frontend_node with those values\n
update_template_values(new_template=frontend_node,\n
previous_template=current_frontend_node["template"])\n                return frontend_node\n\n        def\n
_get_fallback_input(self, **kwargs):\n                return DefaultPromptField(**kwargs)\n"
},
"input": {
    "advanced": false,
    "display_name": "input",
    "dynamic": false,
    "field_type": "str",
    "fileTypes": [],

```

```

    "file_path": "",
    "info": "",
    "input_types": [
        "Message"
    ],
    "list": false,
    "load_from_db": false,
    "multiline": true,
    "name": "input",
    "placeholder": "",
    "required": false,
    "show": true,
    "title_case": false,
    "type": "str",
    "value": ""
},
"template": {
    "_input_type": "PromptInput",
    "advanced": false,
    "display_name": "Template",
    "dynamic": false,
    "info": "",
    "list": false,
    "list_add_label": "Add More",
    "name": "template",
    "override_skip": false,
    "placeholder": "",
    "required": false,
    "show": true,
    "title_case": false,
    "tool_mode": false,
    "trace_as_input": true,
    "track_in_telemetry": false,
    "type": "prompt",
    "value": "You are a workflow classification assistant for a proof-of-concept demo.\n\nYour
task is to classify the user's request into exactly one supported category.\n\nSupported
categories:\n1. bank-collector_settled\n2. bank-transaction_rejected\n3. bank-transaction_settled\n4.
bank-details_found\n5. bank-details_not-found\n6. bank-details_requested\n7.
CREDIT-CARD-STL-FEES_RECEIVED\n8. CREDIT-CARD-STL-ITEM_RECEIVED\n9.
CREDIT-CARD-STL-PAYOUT_RECEIVED\n\nUser request:\n{input}\n"
},
"tool_placeholder": {
    "_input_type": "MessageTextInput",
    "advanced": true,
    "display_name": "Tool Placeholder",
    "dynamic": false,

```

```

    "info": "A placeholder input for tool mode.",
    "input_types": [
      "Message"
    ],
    "list": false,
    "list_add_label": "Add More",
    "load_from_db": false,
    "name": "tool_placeholder",
    "override_skip": false,
    "placeholder": "",
    "required": false,
    "show": false,
    "title_case": false,
    "tool_mode": true,
    "trace_as_input": true,
    "trace_as_metadata": true,
    "track_in_telemetry": false,
    "type": "str",
    "value": ""
  },
  "use_double_brackets": {
    "_input_type": "BoolInput",
    "advanced": true,
    "display_name": "Use Double Brackets",
    "dynamic": false,
    "info": "Use {{variable}} syntax instead of {variable}.",
    "list": false,
    "list_add_label": "Add More",
    "name": "use_double_brackets",
    "override_skip": false,
    "placeholder": "",
    "real_time_refresh": true,
    "required": false,
    "show": true,
    "title_case": false,
    "tool_mode": false,
    "trace_as_metadata": true,
    "track_in_telemetry": true,
    "type": "bool",
    "value": false
  }
},
"tool_mode": false
},
"showNode": true,
"type": "Prompt Template"

```

```

},
"dragging": false,
"id": "Prompt Template-AVLhB",
"measured": {
  "height": 291,
  "width": 320
},
"position": {
  "x": 253,
  "y": 227.5
},
"selected": false,
"type": "genericNode"
},
{
  "data": {
    "id": "Prompt Template-pxOTt",
    "node": {
      "base_classes": [
        "Message"
      ],
      "beta": false,
      "conditional_paths": [],
      "custom_fields": {
        "template": [
          "input",
          "classifier_output"
        ]
      },
      "description": "Create a prompt template with dynamic variables.",
      "display_name": "Prompt Template",
      "documentation": "https://docs.langflow.org/components-prompts",
      "edited": false,
      "error": null,
      "field_order": [
        "template",
        "use_double_brackets",
        "tool_placeholder"
      ],
      "frozen": false,
      "full_path": null,
      "icon": "prompts",
      "is_composition": null,
      "is_input": null,
      "is_output": null,
      "legacy": false,

```

```
"metadata": {
  "code_hash": "3d3fd7b8a36f",
  "dependencies": {
    "dependencies": [
      {
        "name": "lfx",
        "version": null
      }
    ],
    "total_dependencies": 1
  },
  "module": "lfx.components.models_and_agents.prompt.PromptComponent"
},
"minimized": false,
"name": "",
"output_types": [],
"outputs": [
  {
    "allows_loop": false,
    "cache": true,
    "display_name": "Prompt",
    "group_outputs": false,
    "hidden": null,
    "loop_types": null,
    "method": "build_prompt",
    "name": "prompt",
    "options": null,
    "required_inputs": null,
    "selected": "Message",
    "tool_mode": true,
    "types": [
      "Message"
    ],
    "value": "__UNDEFINED__"
  }
],
"pinned": false,
"priority": null,
"replacement": null,
"template": {
  "_type": "Component",
  "classifier_output": {
    "advanced": false,
    "display_name": "classifier_output",
    "dynamic": false,
    "field_type": "str",
```

```

"fileTypes": [],
"file_path": "",
"info": "",
"input_types": [
  "Message"
],
"list": false,
"load_from_db": false,
"multiline": true,
"name": "classifier_output",
"placeholder": "",
"required": false,
"show": true,
"title_case": false,
"type": "str",
"value": ""
},
"code": {
  "advanced": true,
  "dynamic": true,
  "fileTypes": [],
  "file_path": "",
  "info": "",
  "list": false,
  "load_from_db": false,
  "multiline": true,
  "name": "code",
  "password": false,
  "placeholder": "",
  "required": true,
  "show": true,
  "title_case": false,
  "type": "code",
  "value": "from typing import Any\n\nfrom lfx.base.prompts.api_utils import\nprocess_prompt_template\nfrom lfx.custom.custom_component.component import\nComponent\nfrom lfx.inputs.input_mixin import FieldTypes\nfrom lfx.inputs.inputs import\nDefaultPromptField\nfrom lfx.io import BoolInput, MessageTextInput, Output, PromptInput\nfrom\nlfx.log.logger import logger\nfrom lfx.schema.dotdict import dotdict\nfrom lfx.schema.message import\nMessage\nfrom lfx.template.utils import update_template_values\nfrom lfx.utils.mustache_security\nimport validate_mustache_template\n\n\nclass PromptComponent(Component):\n    display_name:\nstr = \"Prompt Template\"\n    description: str = \"Create a prompt template with dynamic\nvariables.\"\n    documentation: str = \"https://docs.langflow.org/components-prompts\"\n    icon =\n\"prompts\"\n    trace_type = \"prompt\"\n    name = \"Prompt Template\"\n    inputs = [\nPromptInput(name=\"template\", display_name=\"Template\"),\n    BoolInput(\nname=\"use_double_brackets\", display_name=\"Use Double Brackets\", \nvalue=False,\n        advanced=True,\n        info=\"Use {{variable}} syntax instead of

```



```

{variable}.\",\n        real_time_refresh=True,\n    ),\n    MessageTextInput(\n
name=\"tool_placeholder\",\n        display_name=\"Tool Placeholder\",\n        tool_mode=True,\n
advanced=True,\n        show=False,\n        info=\"A placeholder input for tool mode.\",\n    ),\n
)\n\n    outputs = [\n        Output(display_name=\"Prompt\", name=\"prompt\",
method=\"build_prompt\"),\n    ]\n\n    def update_build_config(self, build_config: dict, field_value:
Any, field_name: str | None = None) -> dict:\n        \"\"\"Update the template field type based on
the selected mode.\"\"\"\n        if field_name == \"use_double_brackets\":\n            # Change the
template field type based on mode\n            is_mustache = field_value is True\n            if
is_mustache:\n                build_config[\"template\"][\"type\"] =
FieldTypes.MUSTACHE_PROMPT.value\n            else:\n                build_config[\"template\"][\"type\"]
= FieldTypes.PROMPT.value\n            # Re-process the template to update variables when mode
changes\n            template_value = build_config.get(\"template\", {}).get(\"value\", \"\")\n            if
template_value:\n                # Ensure custom_fields is properly initialized\n                if
\"custom_fields\" not in build_config:\n                    build_config[\"custom_fields\"] = {}\n                    #
Clean up fields from the OLD mode before processing with NEW mode\n                    # This ensures
we don't keep fields with wrong syntax even if validation fails\n                    old_custom_fields =
build_config[\"custom_fields\"].get(\"template\", [])\n                    for old_field in
list(old_custom_fields):\n                        # Remove the field from custom_fields and template\n
if old_field in old_custom_fields:\n                            old_custom_fields.remove(old_field)\n
build_config.pop(old_field, None)\n                    # Try to process template with new mode to add new
variables\n                    # If validation fails, at least we cleaned up old fields\n                    try:\n
# Validate mustache templates for security\n                        if is_mustache:\n
                            validate_mustache_template(template_value)\n                            # Re-process template with new mode
to add new variables\n                            _ = process_prompt_template(\n
template=template_value,\n                                name=\"template\",\n
custom_fields=build_config[\"custom_fields\"],\n
frontend_node_template=build_config,\n                                is_mustache=is_mustache,\n                            )\n
except ValueError as e:\n                            # If validation fails, we still updated the mode and cleaned old
fields\n                            # User will see error when they try to save\n
logger.debug(f\"Template validation failed during mode switch: {e}\")\n                            return build_config\n\n
async def build_prompt(self) -> Message:\n        use_double_brackets = self.use_double_brackets if
hasattr(self, \"use_double_brackets\") else False\n        template_format = \"mustache\" if
use_double_brackets else \"f-string\"\n        prompt = await
Message.from_template_and_variables(template_format=template_format, **self._attributes)\n
self.status = prompt.text\n        return prompt\n\n    def _update_template(self, frontend_node:
dict):\n        prompt_template = frontend_node[\"template\"][\"template\"][\"value\"]\n
        use_double_brackets = frontend_node[\"template\"].get(\"use_double_brackets\", {}).get(\"value\",
False)\n        is_mustache = use_double_brackets is True\n        try:\n            # Validate mustache
templates for security\n            if is_mustache:\n
                validate_mustache_template(prompt_template)\n            custom_fields =
frontend_node[\"custom_fields\"]\n            frontend_node_template = frontend_node[\"template\"]\n
            _ = process_prompt_template(\n                template=prompt_template,\n
name=\"template\",\n                custom_fields=custom_fields,\n
frontend_node_template=frontend_node_template,\n                is_mustache=is_mustache,\n
            )\n        except ValueError as e:\n            # If validation fails, don't add variables but allow component
to be created\n            logger.debug(f\"Template validation failed in _update_template: {e}\")

```

```

return frontend_node\n\n    async def update_frontend_node(self, new_frontend_node: dict,
current_frontend_node: dict):\n        \"\"\"This function is called after the code validation is
done.\"\"\"\n        frontend_node = await super().update_frontend_node(new_frontend_node,
current_frontend_node)\n        template = frontend_node[\"template\"]\n        use_double_brackets = frontend_node[\"template\"].get(\"use_double_brackets\", {}).get(\"value\",
False)\n        is_mustache = use_double_brackets is True\n        try:\n            # Validate mustache
templates for security\n            if is_mustache:\n                validate_mustache_template(template)\n\n        # Kept it duplicated for backwards compatibility\n        _ = process_prompt_template(\n
template=template,\n            name=\"template\", \n
custom_fields=frontend_node[\"custom_fields\"], \n
frontend_node_template=frontend_node[\"template\"], \n            is_mustache=is_mustache, \n
)\n        except ValueError as e:\n            # If validation fails, don't add variables but allow component
to be updated\n            logger.debug(f\"Template validation failed in update_frontend_node: {e}\")\n\n        # Now that template is updated, we need to grab any values that were set in the
current_frontend_node\n        # and update the frontend_node with those values\n        update_template_values(new_template=frontend_node,
previous_template=current_frontend_node[\"template\"])\n        return frontend_node\n\n    def
_get_fallback_input(self, **kwargs):\n        return DefaultPromptField(**kwargs)\n"
},
    "input": {
        "advanced": false,
        "display_name": "input",
        "dynamic": false,
        "field_type": "str",
        "fileTypes": [],
        "file_path": "",
        "info": "",
        "input_types": [
            "Message"
        ],
        "list": false,
        "load_from_db": false,
        "multiline": true,
        "name": "input",
        "placeholder": "",
        "required": false,
        "show": true,
        "title_case": false,
        "type": "str",
        "value": ""
    },
    "template": {
        "_input_type": "PromptInput",
        "advanced": false,
        "display_name": "Template",
        "dynamic": false,

```

```
"info": "",
"list": false,
"list_add_label": "Add More",
"name": "template",
"override_skip": false,
"placeholder": "",
"required": false,
"show": true,
"title_case": false,
"tool_mode": false,
"trace_as_input": true,
"track_in_telemetry": false,
"type": "prompt",
"value": "You are generating a client-facing proof-of-concept response.\n\nOriginal user request:\n{input}\n\nClassifier result:\n{classifier_output}\n\nGenerate a polished response with the following sections:\n\n1. Request Summary\n2. Selected Template Category\n3. Demo Response\n4. Sample Structured Output\n5. Business Value\n\nRules:\n• Do not mention internal code, hidden logic, browser extension details, or proprietary implementation.\n• Keep the tone professional and presentation-friendly.\n• The Sample Structured Output must be valid JSON.\n• Keep the response concise but clear."
```

```
},
"tool_placeholder": {
  "_input_type": "MessageTextInput",
  "advanced": true,
  "display_name": "Tool Placeholder",
  "dynamic": false,
  "info": "A placeholder input for tool mode.",
  "input_types": [
    "Message"
  ],
  "list": false,
  "list_add_label": "Add More",
  "load_from_db": false,
  "name": "tool_placeholder",
  "override_skip": false,
  "placeholder": "",
  "required": false,
  "show": false,
  "title_case": false,
  "tool_mode": true,
  "trace_as_input": true,
  "trace_as_metadata": true,
  "track_in_telemetry": false,
  "type": "str",
  "value": ""
},
```

```

    "use_double_brackets": {
      "_input_type": "BoolInput",
      "advanced": true,
      "display_name": "Use Double Brackets",
      "dynamic": false,
      "info": "Use {{variable}} syntax instead of {variable}.",
      "list": false,
      "list_add_label": "Add More",
      "name": "use_double_brackets",
      "override_skip": false,
      "placeholder": "",
      "real_time_refresh": true,
      "required": false,
      "show": true,
      "title_case": false,
      "tool_mode": false,
      "trace_as_metadata": true,
      "track_in_telemetry": true,
      "type": "bool",
      "value": false
    }
  },
  "tool_mode": false
},
"showNode": true,
"type": "Prompt Template"
},
"dragging": false,
"id": "Prompt Template-pxOTt",
"measured": {
  "height": 373,
  "width": 320
},
"position": {
  "x": 954,
  "y": 227.5
},
"selected": true,
"type": "genericNode"
},
{
  "data": {
    "id": "GroqModel-AjfvL",
    "node": {
      "base_classes": [
        "LanguageModel",

```

```
    "Message"
  ],
  "beta": false,
  "conditional_paths": [],
  "custom_fields": {},
  "description": "Generate text using Groq.",
  "display_name": "Groq",
  "documentation": "",
  "edited": false,
  "field_order": [
    "input_value",
    "system_message",
    "stream",
    "api_key",
    "base_url",
    "max_tokens",
    "temperature",
    "n",
    "model_name",
    "tool_model_enabled"
  ],
  "frozen": false,
  "icon": "Groq",
  "last_updated": "2026-05-08T06:35:58.370Z",
  "legacy": false,
  "lf_version": "1.9.1",
  "metadata": {
    "code_hash": "8a55d3fa8173",
    "dependencies": {
      "dependencies": [
        {
          "name": "pydantic",
          "version": "2.12.5"
        },
        {
          "name": "lfx",
          "version": null
        },
        {
          "name": "langchain_groq",
          "version": "1.1.2"
        }
      ]
    },
    "total_dependencies": 3
  },
  "keywords": [
```

```

    "model",
    "llm",
    "language model",
    "large language model"
  ],
  "module": "lfx.components.groq.groq.GroqModel"
},
"minimized": false,
"output_types": [],
"outputs": [
  {
    "allows_loop": false,
    "cache": true,
    "display_name": "Model Response",
    "group_outputs": false,
    "loop_types": null,
    "method": "text_response",
    "name": "text_output",
    "options": null,
    "required_inputs": null,
    "selected": "Message",
    "tool_mode": true,
    "types": [
      "Message"
    ],
    "value": "__UNDEFINED__"
  },
  {
    "allows_loop": false,
    "cache": true,
    "display_name": "Language Model",
    "group_outputs": false,
    "loop_types": null,
    "method": "build_model",
    "name": "model_output",
    "options": null,
    "required_inputs": null,
    "selected": "LanguageModel",
    "tool_mode": true,
    "types": [
      "LanguageModel"
    ],
    "value": "__UNDEFINED__"
  }
],
"pinned": false,

```

```
"template": {
  "_frontend_node_flow_id": {
    "value": "efb6bca2-9013-4f40-928f-48711256f151"
  },
  "_frontend_node_folder_id": {
    "value": "f36d5528-8d1e-4e3e-abe0-81b2ce260523"
  },
  "_type": "Component",
  "api_key": {
    "_input_type": "SecretStrInput",
    "advanced": false,
    "display_name": "Groq API Key",
    "dynamic": false,
    "info": "API key for the Groq API.",
    "input_types": [],
    "load_from_db": true,
    "name": "api_key",
    "override_skip": false,
    "password": true,
    "placeholder": "",
    "real_time_refresh": true,
    "required": false,
    "show": true,
    "title_case": false,
    "track_in_telemetry": false,
    "type": "str",
    "value": "langflowAPIKey"
  },
  "base_url": {
    "_input_type": "MessageTextInput",
    "advanced": true,
    "display_name": "Groq API Base",
    "dynamic": false,
    "info": "Base URL path for API requests, leave blank if not using a proxy or service
emulator.",
    "input_types": [
      "Message"
    ],
    "list": false,
    "list_add_label": "Add More",
    "load_from_db": false,
    "name": "base_url",
    "override_skip": false,
    "placeholder": "",
    "real_time_refresh": true,
    "required": false,
```

```

        "show": true,
        "title_case": false,
        "tool_mode": false,
        "trace_as_input": true,
        "trace_as_metadata": true,
        "track_in_telemetry": false,
        "type": "str",
        "value": "https://api.groq.com"
    },
    "code": {
        "advanced": true,
        "dynamic": true,
        "fileTypes": [],
        "file_path": "",
        "info": "",
        "list": false,
        "load_from_db": false,
        "multiline": true,
        "name": "code",
        "password": false,
        "placeholder": "",
        "required": true,
        "show": true,
        "title_case": false,
        "type": "code",
        "value": "from pydantic.v1 import SecretStr\n\nfrom lfx.base.models.groq_constants import\nGROQ_MODELS\nfrom lfx.base.models.groq_model_discovery import get_groq_models\nfrom\nlfx.base.models.model import LCModelComponent\nfrom lfx.field_typing import\nLanguageModel\nfrom lfx.field_typing.range_spec import RangeSpec\nfrom lfx.io import BoolInput,\nDropdownInput, IntInput, MessageTextInput, SecretStrInput, SliderInput\nfrom lfx.log.logger import\nlogger\n\n\nclass GroqModel(LCModelComponent):\n    display_name: str = \"Groq\"\n    description:\nstr = \"Generate text using Groq.\"\n    icon = \"Groq\"\n    name = \"GroqModel\"\n\n    inputs = [\n        *LCModelComponent.get_base_inputs(),\n        SecretStrInput(\n            name=\"api_key\",\n            display_name=\"Groq API Key\", info=\"API key for the Groq API.\", real_time_refresh=True\n        ),\n        MessageTextInput(\n            name=\"base_url\", display_name=\"Groq API\nBase\", info=\"Base URL path for API requests, leave blank if not using a proxy or service\nemulator.\",\n            advanced=True, value=\"https://api.groq.com\", real_time_refresh=True\n        ),\n        IntInput(\n            name=\"max_tokens\", display_name=\"Max Output Tokens\", info=\"The maximum number of tokens to\ngenerate.\",\n            advanced=True\n        ),\n        SliderInput(\n            name=\"temperature\", display_name=\"Temperature\", value=0.1, info=\"Run inference with this\ntemperature. Must be in the closed interval [0.0, 1.0].\", range_spec=RangeSpec(min=0,\nmax=1, step=0.01),\n            advanced=True\n        ),\n        IntInput(\n            name=\"n\", display_name=\"N\", info=\"Number of chat completions to generate for each prompt.\"\n\n            \"Note that the API may not return the full n completions if duplicates are generated.\",\n            advanced=True\n        ),\n        DropdownInput(\n            name=\"model_name\",

```



```

display_name="Model",\n      info="The name of the model to use. Add your Groq API key to
access additional available models.",\n      options=GROQ_MODELS,\n
value=GROQ_MODELS[0],\n      refresh_button=True,\n      combobox=True,\n      ),\n
BoolInput(\n      name="tool_model_enabled",\n      display_name="Enable Tool Models",\n
info=(\n      "Select if you want to use models that can work with tools. If yes, only those
models will be shown.",\n      ),\n      advanced=False,\n      value=False,\n
real_time_refresh=True,\n      ),\n      ]\n\n def get_models(self, *, tool_model_enabled: bool | None
= None) -> list[str]:\n      """Get available Groq models using the dynamic discovery system.\n\n
This method uses the groq_model_discovery module which:\n      - Fetches models directly from
Groq API\n      - Automatically tests tool calling support\n      - Caches results for 24 hours\n      -
Falls back to hardcoded list if API fails\n\n      Args:\n      tool_model_enabled: If True, only
return models that support tool calling\n\n      Returns:\n      List of available model IDs\n
"""\n      try:\n      # Get models with metadata from dynamic discovery system\n
api_key = self.api_key if hasattr(self, "api_key") and self.api_key else None\n
models_metadata = get_groq_models(api_key=api_key)\n      # Filter out non-LLM models
(audio, TTS, guards)\n      model_ids = [\n      model_id for model_id, metadata in
models_metadata.items() if not metadata.get("not_supported", False)\n      ]\n      # Filter
by tool calling support if requested\n      if tool_model_enabled:\n      model_ids =
[model_id for model_id in model_ids if models_metadata[model_id].get("tool_calling", False)]\n
logger.info(f"Loaded {len(model_ids)} Groq models with tool calling support")\n      else:\n
logger.info(f"Loaded {len(model_ids)} Groq models")\n      except (ValueError, KeyError, TypeError,
ImportError):\n      logger.exception("Error getting model names")\n      # Fallback to
hardcoded list from groq_constants.py\n      return GROQ_MODELS\n      else:\n      return
model_ids\n\n def update_build_config(self, build_config: dict, field_value: str, field_name: str |
None = None):\n      if field_name in {"base_url", "model_name", "tool_model_enabled",
"api_key"} and field_value:\n      try:\n      if len(self.api_key) != 0:\n      try:\n
ids = self.get_models(tool_model_enabled=self.tool_model_enabled)\n      except
(ValueError, KeyError, TypeError, ImportError):\n      logger.exception("Error getting
model names")\n      ids = GROQ_MODELS\n      ids = ids or
GROQ_MODELS\n      build_config.setdefault("model_name", {})\n
build_config["model_name"]["options"] = ids\n
build_config["model_name"].setdefault("value", ids[0])\n      except (ValueError, KeyError,
TypeError, AttributeError) as e:\n      msg = f"Error getting model names: {e}"\n      raise
ValueError(msg) from e\n      return build_config\n\n def build_model(self) -> LanguageModel: #
type: ignore[type-var]\n      try:\n      from langchain_groq import ChatGroq\n      except
ImportError as e:\n      msg = "langchain-groq is not installed. Please install it with `pip install
langchain-groq`." \n      raise ImportError(msg) from e\n      return ChatGroq(\n
model=self.model_name,\n      max_tokens=self.max_tokens or None,\n
temperature=self.temperature,\n      base_url=self.base_url,\n      n=self.n or 1,\n
api_key=SecretStr(self.api_key).get_secret_value(),\n      streaming=self.stream,\n      )\n
},\n
"input_value": {\n
"_input_type": "MessageInput",\n
"advanced": false,\n
"display_name": "Input",\n
"dynamic": false,\n

```

```
"info": "",
"input_types": [
  "Message"
],
"list": false,
"list_add_label": "Add More",
"load_from_db": false,
"name": "input_value",
"override_skip": false,
"placeholder": "",
"required": false,
"show": true,
"title_case": false,
"tool_mode": false,
"trace_as_input": true,
"trace_as_metadata": true,
"track_in_telemetry": false,
"type": "str",
"value": ""
},
"is_refresh": false,
"max_tokens": {
  "_input_type": "IntInput",
  "advanced": true,
  "display_name": "Max Output Tokens",
  "dynamic": false,
  "info": "The maximum number of tokens to generate.",
  "list": false,
  "list_add_label": "Add More",
  "name": "max_tokens",
  "override_skip": false,
  "placeholder": "",
  "required": false,
  "show": true,
  "title_case": false,
  "tool_mode": false,
  "trace_as_metadata": true,
  "track_in_telemetry": true,
  "type": "int",
  "value": 0
},
"model_name": {
  "_input_type": "DropdownInput",
  "advanced": false,
  "combobox": true,
  "dialog_inputs": {},
```

```

    "display_name": "Model",
    "dynamic": false,
    "external_options": {},
    "info": "The name of the model to use. Add your Groq API key to access additional
available models.",
    "name": "model_name",
    "options": [
        "openai/gpt-oss-120b",
        "llama-3.1-8b-instant",
        "qwen/qwen3-32b",
        "llama-3.3-70b-versatile",
        "meta-llama/llama-4-scout-17b-16e-instruct",
        "groq/compound-mini",
        "openai/gpt-oss-20b",
        "groq/compound",
        "allam-2-7b"
    ],
    "options_metadata": [],
    "override_skip": false,
    "placeholder": "",
    "refresh_button": true,
    "required": false,
    "show": true,
    "title_case": false,
    "toggle": false,
    "tool_mode": false,
    "trace_as_metadata": true,
    "track_in_telemetry": true,
    "type": "str",
    "value": "openai/gpt-oss-120b"
},
    "n": {
        "_input_type": "IntInput",
        "advanced": true,
        "display_name": "N",
        "dynamic": false,
        "info": "Number of chat completions to generate for each prompt. Note that the API may
not return the full n completions if duplicates are generated.",
        "list": false,
        "list_add_label": "Add More",
        "name": "n",
        "override_skip": false,
        "placeholder": "",
        "required": false,
        "show": true,
        "title_case": false,

```

```
"tool_mode": false,
"trace_as_metadata": true,
"track_in_telemetry": true,
"type": "int",
"value": 0
},
"stream": {
  "_input_type": "BoolInput",
  "advanced": true,
  "display_name": "Stream",
  "dynamic": false,
  "info": "Stream the response from the model. Streaming works only in Chat.",
  "list": false,
  "list_add_label": "Add More",
  "name": "stream",
  "override_skip": false,
  "placeholder": "",
  "required": false,
  "show": true,
  "title_case": false,
  "tool_mode": false,
  "trace_as_metadata": true,
  "track_in_telemetry": true,
  "type": "bool",
  "value": false
},
"system_message": {
  "_input_type": "MultilineInput",
  "advanced": false,
  "ai_enabled": false,
  "copy_field": false,
  "display_name": "System Message",
  "dynamic": false,
  "info": "System message to pass to the model.",
  "input_types": [
    "Message"
  ],
  "list": false,
  "list_add_label": "Add More",
  "load_from_db": false,
  "multiline": true,
  "name": "system_message",
  "override_skip": false,
  "password": false,
  "placeholder": "",
  "required": false,
```

```

    "show": true,
    "title_case": false,
    "tool_mode": false,
    "trace_as_input": true,
    "trace_as_metadata": true,
    "track_in_telemetry": false,
    "type": "str",
    "value": ""
  },
  "temperature": {
    "_input_type": "SliderInput",
    "advanced": true,
    "display_name": "Temperature",
    "dynamic": false,
    "info": "Run inference with this temperature. Must be in the closed interval [0.0, 1.0].",
    "max_label": "",
    "max_label_icon": "",
    "min_label": "",
    "min_label_icon": "",
    "name": "temperature",
    "override_skip": false,
    "placeholder": "",
    "range_spec": {
      "max": 1,
      "min": 0,
      "step": 0.01,
      "step_type": "float"
    },
    "required": false,
    "show": true,
    "slider_buttons": false,
    "slider_buttons_options": [],
    "slider_input": false,
    "title_case": false,
    "tool_mode": false,
    "track_in_telemetry": false,
    "type": "slider",
    "value": 0.1
  },
  "tool_model_enabled": {
    "_input_type": "BoolInput",
    "advanced": false,
    "display_name": "Enable Tool Models",
    "dynamic": false,
    "info": "Select if you want to use models that can work with tools. If yes, only those models
will be shown.",

```

```

        "list": false,
        "list_add_label": "Add More",
        "name": "tool_model_enabled",
        "override_skip": false,
        "placeholder": "",
        "real_time_refresh": true,
        "required": false,
        "show": true,
        "title_case": false,
        "tool_mode": false,
        "trace_as_metadata": true,
        "track_in_telemetry": true,
        "type": "bool",
        "value": false
    }
},
    "tool_mode": false
},
    "selected_output": "text_output",
    "showNode": true,
    "type": "GroqModel"
},
    "dragging": false,
    "id": "GroqModel-AjfvL",
    "measured": {
        "height": 495,
        "width": 320
    },
    "position": {
        "x": 609,
        "y": 222.5
    },
    "selected": false,
    "type": "genericNode"
},
{
    "data": {
        "id": "GroqModel-Zbm2F",
        "node": {
            "base_classes": [
                "LanguageModel",
                "Message"
            ],
            "beta": false,
            "conditional_paths": [],
            "custom_fields": {},

```

```
"description": "Generate text using Groq.",
"display_name": "Groq",
"documentation": "",
"edited": false,
"field_order": [
  "input_value",
  "system_message",
  "stream",
  "api_key",
  "base_url",
  "max_tokens",
  "temperature",
  "n",
  "model_name",
  "tool_model_enabled"
],
"frozen": false,
"icon": "Groq",
"last_updated": "2026-05-08T06:36:10.357Z",
"legacy": false,
"lf_version": "1.9.1",
"metadata": {
  "code_hash": "8a55d3fa8173",
  "dependencies": {
    "dependencies": [
      {
        "name": "pydantic",
        "version": "2.12.5"
      },
      {
        "name": "lfx",
        "version": null
      },
      {
        "name": "langchain_groq",
        "version": "1.1.2"
      }
    ]
  },
  "total_dependencies": 3
},
"keywords": [
  "model",
  "llm",
  "language model",
  "large language model"
],
```

```

    "module": "lfx.components.groq.groq.GroqModel"
  },
  "minimized": false,
  "output_types": [],
  "outputs": [
    {
      "allows_loop": false,
      "cache": true,
      "display_name": "Model Response",
      "group_outputs": false,
      "loop_types": null,
      "method": "text_response",
      "name": "text_output",
      "options": null,
      "required_inputs": null,
      "selected": "Message",
      "tool_mode": true,
      "types": [
        "Message"
      ],
      "value": "__UNDEFINED__"
    },
    {
      "allows_loop": false,
      "cache": true,
      "display_name": "Language Model",
      "group_outputs": false,
      "loop_types": null,
      "method": "build_model",
      "name": "model_output",
      "options": null,
      "required_inputs": null,
      "selected": "LanguageModel",
      "tool_mode": true,
      "types": [
        "LanguageModel"
      ],
      "value": "__UNDEFINED__"
    }
  ],
  "pinned": false,
  "template": {
    "_frontend_node_flow_id": {
      "value": "efb6bca2-9013-4f40-928f-48711256f151"
    },
    "_frontend_node_folder_id": {

```



```
    "value": "f36d5528-8d1e-4e3e-abe0-81b2ce260523"
  },
  "_type": "Component",
  "api_key": {
    "_input_type": "SecretStrInput",
    "advanced": false,
    "display_name": "Groq API Key",
    "dynamic": false,
    "info": "API key for the Groq API.",
    "input_types": [],
    "load_from_db": true,
    "name": "api_key",
    "override_skip": false,
    "password": true,
    "placeholder": "",
    "real_time_refresh": true,
    "required": false,
    "show": true,
    "title_case": false,
    "track_in_telemetry": false,
    "type": "str",
    "value": "langflowAPIKey"
  },
  "base_url": {
    "_input_type": "MessageTextInput",
    "advanced": true,
    "display_name": "Groq API Base",
    "dynamic": false,
    "info": "Base URL path for API requests, leave blank if not using a proxy or service
emulator.",
    "input_types": [
      "Message"
    ],
    "list": false,
    "list_add_label": "Add More",
    "load_from_db": false,
    "name": "base_url",
    "override_skip": false,
    "placeholder": "",
    "real_time_refresh": true,
    "required": false,
    "show": true,
    "title_case": false,
    "tool_mode": false,
    "trace_as_input": true,
    "trace_as_metadata": true,
```

```

        "track_in_telemetry": false,
        "type": "str",
        "value": "https://api.groq.com"
    },
    "code": {
        "advanced": true,
        "dynamic": true,
        "fileTypes": [],
        "file_path": "",
        "info": "",
        "list": false,
        "load_from_db": false,
        "multiline": true,
        "name": "code",
        "password": false,
        "placeholder": "",
        "required": true,
        "show": true,
        "title_case": false,
        "type": "code",
        "value": "from pydantic.v1 import SecretStr\n\nfrom lfx.base.models.groq_constants import\nGROQ_MODELS\n\nfrom lfx.base.models.groq_model_discovery import get_groq_models\n\nfrom\nlfx.base.models.model import LCModelComponent\n\nfrom lfx.field_typing import\nLanguageModel\n\nfrom lfx.field_typing.range_spec import RangeSpec\n\nfrom lfx.io import BoolInput,\nDropdownInput, IntInput, MessageTextInput, SecretStrInput, SliderInput\n\nfrom lfx.log.logger import\nlogger\n\n\nclass GroqModel(LCModelComponent):\n    display_name: str = \"Groq\"\n    description:\nstr = \"Generate text using Groq.\"\n    icon = \"Groq\"\n    name = \"GroqModel\"\n    inputs = [\n        *LCModelComponent.get_base_inputs(),\n        SecretStrInput(\n            name=\"api_key\",\n            display_name=\"Groq API Key\", info=\"API key for the Groq API.\", real_time_refresh=True\n        ),\n        MessageTextInput(\n            name=\"base_url\", display_name=\"Groq API\nBase\", info=\"Base URL path for API requests, leave blank if not using a proxy or service\nemulator.\",\n            advanced=True, value=\"https://api.groq.com\", real_time_refresh=True\n        ),\n        IntInput(\n            name=\"max_tokens\", display_name=\"Max Output Tokens\", info=\"The maximum number of tokens to\n        generate.\",\n            advanced=True\n        ),\n        SliderInput(\n            name=\"temperature\", display_name=\"Temperature\", value=0.1, info=\"Run inference with this\ntemperature. Must be in the closed interval [0.0, 1.0].\", range_spec=RangeSpec(min=0,\n        max=1, step=0.01),\n            advanced=True\n        ),\n        IntInput(\n            name=\"n\", display_name=\"N\", info=\"Number of chat completions to generate for each prompt.\"\n        \"Note that the API may not return the full n completions if duplicates are generated.\",\n            advanced=True\n        ),\n        DropdownInput(\n            name=\"model_name\", display_name=\"Model\", info=\"The name of the model to use. Add your Groq API key to\n        access additional available models.\", options=GROQ_MODELS, value=GROQ_MODELS[0],\n            refresh_button=True, combobox=True\n        ),\n        BoolInput(\n            name=\"tool_model_enabled\", display_name=\"Enable Tool Models\", info=\"\n        \"Select if you want to use models that can work with tools. If yes, only those

```

```

models will be shown.\n        ),\n        advanced=False,\n        value=False,\n        real_time_refresh=True,\n        ],\n    ]\n\n    def get_models(self, *, tool_model_enabled: bool | None\n    = None) -> list[str]:\n        """Get available Groq models using the dynamic discovery system.\n        This method uses the groq_model_discovery module which:\n        - Fetches models directly from Groq API\n        - Automatically tests tool calling support\n        - Caches results for 24 hours\n        - Falls back to hardcoded list if API fails\n\n        Args:\n            tool_model_enabled: If True, only return models that support tool calling\n\n        Returns:\n            List of available model IDs\n\n        """\n        try:\n            # Get models with metadata from dynamic discovery system\n            api_key = self.api_key if hasattr(self, "api_key") and self.api_key else None\n            models_metadata = get_groq_models(api_key=api_key)\n            # Filter out non-LLM models (audio, TTS, guards)\n            model_ids = [\n                model_id for model_id, metadata in models_metadata.items()\n                if not metadata.get("not_supported", False)\n            ]\n            # Filter by tool calling support if requested\n            if tool_model_enabled:\n                model_ids = [\n                    model_id for model_id in model_ids\n                    if models_metadata[model_id].get("tool_calling", False)\n                ]\n            logger.info(f"Loaded {len(model_ids)} Groq models with tool calling support")\n            else:\n                logger.info(f"Loaded {len(model_ids)} Groq models")\n        except (ValueError, KeyError, TypeError, ImportError):\n            logger.exception("Error getting model names")\n            # Fallback to hardcoded list from groq_constants.py\n            return GROQ_MODELS\n        else:\n            return model_ids\n\n    def update_build_config(self, build_config: dict, field_value: str, field_name: str | None = None):\n        if field_name in {"base_url", "model_name", "tool_model_enabled", "api_key"} and field_value:\n            try:\n                if len(self.api_key) != 0:\n                    try:\n                        ids = self.get_models(tool_model_enabled=self.tool_model_enabled)\n                    except (ValueError, KeyError, TypeError, ImportError):\n                        logger.exception("Error getting model names")\n                    ids = GROQ_MODELS\n                    ids = ids or GROQ_MODELS\n                    build_config.setdefault("model_name", {})\n                    build_config["model_name"]["options"] = ids\n                    build_config["model_name"].setdefault("value", ids[0])\n            except (ValueError, KeyError, TypeError, AttributeError) as e:\n                msg = f"Error getting model names: {e}"\n                raise ValueError(msg) from e\n            return build_config\n\n    def build_model(self) -> LanguageModel: # type: ignore[type-var]\n        try:\n            from langchain_groq import ChatGroq\n        except ImportError as e:\n            msg = "langchain-groq is not installed. Please install it with `pip install langchain-groq`.\n            raise ImportError(msg) from e\n        return ChatGroq(\n            model=self.model_name,\n            max_tokens=self.max_tokens or None,\n            temperature=self.temperature,\n            base_url=self.base_url,\n            n=self.n or 1,\n            api_key=SecretStr(self.api_key).get_secret_value(),\n            streaming=self.stream,\n        )\n\n    },\n    "input_value": {\n        "_input_type": "MessageInput",\n        "advanced": false,\n        "display_name": "Input",\n        "dynamic": false,\n        "info": "",\n        "input_types": [\n            "Message"\n        ],\n    },\n    "list": false,

```

```

    "list_add_label": "Add More",
    "load_from_db": false,
    "name": "input_value",
    "override_skip": false,
    "placeholder": "",
    "required": false,
    "show": true,
    "title_case": false,
    "tool_mode": false,
    "trace_as_input": true,
    "trace_as_metadata": true,
    "track_in_telemetry": false,
    "type": "str",
    "value": ""
  },
  "is_refresh": false,
  "max_tokens": {
    "_input_type": "IntInput",
    "advanced": true,
    "display_name": "Max Output Tokens",
    "dynamic": false,
    "info": "The maximum number of tokens to generate.",
    "list": false,
    "list_add_label": "Add More",
    "name": "max_tokens",
    "override_skip": false,
    "placeholder": "",
    "required": false,
    "show": true,
    "title_case": false,
    "tool_mode": false,
    "trace_as_metadata": true,
    "track_in_telemetry": true,
    "type": "int",
    "value": 0
  },
  "model_name": {
    "_input_type": "DropdownInput",
    "advanced": false,
    "combobox": true,
    "dialog_inputs": {},
    "display_name": "Model",
    "dynamic": false,
    "external_options": {},
    "info": "The name of the model to use. Add your Groq API key to access additional
available models.",

```

```

"name": "model_name",
"options": [
  "openai/gpt-oss-120b",
  "llama-3.1-8b-instant",
  "qwen/qwen3-32b",
  "llama-3.3-70b-versatile",
  "meta-llama/llama-4-scout-17b-16e-instruct",
  "groq/compound-mini",
  "openai/gpt-oss-20b",
  "groq/compound",
  "allam-2-7b"
],
"options_metadata": [],
"override_skip": false,
"placeholder": "",
"refresh_button": true,
"required": false,
"show": true,
"title_case": false,
"toggle": false,
"tool_mode": false,
"trace_as_metadata": true,
"track_in_telemetry": true,
"type": "str",
"value": "openai/gpt-oss-120b"
},
{n": {
  "_input_type": "IntInput",
  "advanced": true,
  "display_name": "N",
  "dynamic": false,
  "info": "Number of chat completions to generate for each prompt. Note that the API may
not return the full n completions if duplicates are generated.",
  "list": false,
  "list_add_label": "Add More",
  "name": "n",
  "override_skip": false,
  "placeholder": "",
  "required": false,
  "show": true,
  "title_case": false,
  "tool_mode": false,
  "trace_as_metadata": true,
  "track_in_telemetry": true,
  "type": "int",
  "value": 0

```

```

},
"stream": {
  "_input_type": "BoolInput",
  "advanced": true,
  "display_name": "Stream",
  "dynamic": false,
  "info": "Stream the response from the model. Streaming works only in Chat.",
  "list": false,
  "list_add_label": "Add More",
  "name": "stream",
  "override_skip": false,
  "placeholder": "",
  "required": false,
  "show": true,
  "title_case": false,
  "tool_mode": false,
  "trace_as_metadata": true,
  "track_in_telemetry": true,
  "type": "bool",
  "value": false
},
"system_message": {
  "_input_type": "MultilineInput",
  "advanced": false,
  "ai_enabled": false,
  "copy_field": false,
  "display_name": "System Message",
  "dynamic": false,
  "info": "System message to pass to the model.",
  "input_types": [
    "Message"
  ],
  "list": false,
  "list_add_label": "Add More",
  "load_from_db": false,
  "multiline": true,
  "name": "system_message",
  "override_skip": false,
  "password": false,
  "placeholder": "",
  "required": false,
  "show": true,
  "title_case": false,
  "tool_mode": false,
  "trace_as_input": true,
  "trace_as_metadata": true,

```

```

    "track_in_telemetry": false,
    "type": "str",
    "value": ""
  },
  "temperature": {
    "_input_type": "SliderInput",
    "advanced": true,
    "display_name": "Temperature",
    "dynamic": false,
    "info": "Run inference with this temperature. Must be in the closed interval [0.0, 1.0].",
    "max_label": "",
    "max_label_icon": "",
    "min_label": "",
    "min_label_icon": "",
    "name": "temperature",
    "override_skip": false,
    "placeholder": "",
    "range_spec": {
      "max": 1,
      "min": 0,
      "step": 0.01,
      "step_type": "float"
    },
    "required": false,
    "show": true,
    "slider_buttons": false,
    "slider_buttons_options": [],
    "slider_input": false,
    "title_case": false,
    "tool_mode": false,
    "track_in_telemetry": false,
    "type": "slider",
    "value": 0.3
  },
  "tool_model_enabled": {
    "_input_type": "BoolInput",
    "advanced": false,
    "display_name": "Enable Tool Models",
    "dynamic": false,
    "info": "Select if you want to use models that can work with tools. If yes, only those models
will be shown.",
    "list": false,
    "list_add_label": "Add More",
    "name": "tool_model_enabled",
    "override_skip": false,
    "placeholder": "",

```

```

    "real_time_refresh": true,
    "required": false,
    "show": true,
    "title_case": false,
    "tool_mode": false,
    "trace_as_metadata": true,
    "track_in_telemetry": true,
    "type": "bool",
    "value": false
  }
},
"tool_mode": false
},
"selected_output": "text_output",
"showNode": true,
"type": "GroqModel"
},
"dragging": false,
"id": "GroqModel-Zbm2F",
"measured": {
  "height": 495,
  "width": 320
},
"position": {
  "x": 1304.1591051590613,
  "y": 223.63561532548977
},
"selected": false,
"type": "genericNode"
},
{
  "data": {
    "id": "ChatOutput-D1HOI",
    "node": {
      "base_classes": [
        "Message"
      ],
      "beta": false,
      "conditional_paths": [],
      "custom_fields": {},
      "description": "Display a chat message in the Playground.",
      "display_name": "Chat Output",
      "documentation": "https://docs.langflow.org/chat-input-and-output",
      "edited": false,
      "field_order": [
        "input_value",

```



```
"should_store_message",
"sender",
"sender_name",
"session_id",
"context_id",
"data_template",
"clean_data"
],
"frozen": false,
"icon": "MessagesSquare",
"legacy": false,
"lf_version": "1.9.1",
"metadata": {
  "code_hash": "84009527d08c",
  "dependencies": {
    "dependencies": [
      {
        "name": "orjson",
        "version": "3.11.8"
      },
      {
        "name": "fastapi",
        "version": "0.136.1"
      },
      {
        "name": "lfx",
        "version": null
      }
    ]
  },
  "total_dependencies": 3
},
"module": "lfx.components.input_output.chat_output.ChatOutput"
},
"minimized": true,
"output_types": [],
"outputs": [
  {
    "allows_loop": false,
    "cache": true,
    "display_name": "Output Message",
    "group_outputs": false,
    "method": "message_response",
    "name": "message",
    "selected": "Message",
    "tool_mode": true,
    "types": [
```

```

      "Message"
    ],
    "value": "__UNDEFINED__"
  }
],
"pinned": false,
"template": {
  "_type": "Component",
  "clean_data": {
    "_input_type": "BoolInput",
    "advanced": true,
    "display_name": "Basic Clean Data",
    "dynamic": false,
    "info": "Whether to clean data before converting to string.",
    "list": false,
    "list_add_label": "Add More",
    "name": "clean_data",
    "override_skip": false,
    "placeholder": "",
    "required": false,
    "show": true,
    "title_case": false,
    "tool_mode": false,
    "trace_as_metadata": true,
    "track_in_telemetry": true,
    "type": "bool",
    "value": true
  },
  "code": {
    "advanced": true,
    "dynamic": true,
    "fileTypes": [],
    "file_path": "",
    "info": "",
    "list": false,
    "load_from_db": false,
    "multiline": true,
    "name": "code",
    "password": false,
    "placeholder": "",
    "required": true,
    "show": true,
    "title_case": false,
    "type": "code",
    "value": "from collections.abc import Generator\nfrom typing import Any\n\nimport\norjson\nfrom fastapi.encoders import jsonable_encoder\n\nfrom lfx.base.io.chat import

```

```

ChatComponent\nfrom lfx.helpers.data import safe_convert\nfrom lfx.inputs.inputs import BoolInput,\nDropdownInput, HandleInput, MessageTextInput\nfrom lfx.schema.data import Data\nfrom\nlfx.schema.dataframe import DataFrame\nfrom lfx.schema.message import Message\nfrom\nlfx.schema.properties import Source\nfrom lfx.template.field.base import Output\nfrom\nlfx.utils.constants import (\n    MESSAGE_SENDER_AI,\n    MESSAGE_SENDER_NAME_AI,\n    MESSAGE_SENDER_USER,\n)\n\n\nclass ChatOutput(ChatComponent):\n    display_name =\n    \"Chat Output\"\n    description = \"Display a chat message in the Playground.\"\n    documentation:\n    str = \"https://docs.langflow.org/chat-input-and-output\"\n    icon = \"MessagesSquare\"\n    name =\n    \"ChatOutput\"\n    minimized = True\n\n    inputs = [\n        HandleInput(\n            name=\"input_value\",\n            display_name=\"Inputs\",\n            info=\"Message to be passed as\n            output.\"",\n            input_types=[\"Data\", \"JSON\", \"DataFrame\", \"Table\", \"Message\"],\n            required=True,\n        ),\n        BoolInput(\n            name=\"should_store_message\",\n            display_name=\"Store Messages\",\n            info=\"Store the message in the history.\"",\n            value=True,\n            advanced=True,\n        ),\n        DropdownInput(\n            name=\"sender\",\n            display_name=\"Sender Type\",\n            options=[MESSAGE_SENDER_AI,\n                MESSAGE_SENDER_USER],\n            value=MESSAGE_SENDER_AI,\n            advanced=True,\n            info=\"Type of sender.\"",\n        ),\n        MessageTextInput(\n            name=\"sender_name\",\n            display_name=\"Sender Name\",\n            info=\"Name of the sender.\"",\n            value=MESSAGE_SENDER_NAME_AI,\n            advanced=True,\n        ),\n        MessageTextInput(\n            name=\"session_id\",\n            display_name=\"Session ID\",\n            info=\"The session ID of the chat. If empty, the current session ID parameter will be used.\"",\n            advanced=True,\n        ),\n        MessageTextInput(\n            name=\"context_id\",\n            display_name=\"Context ID\",\n            info=\"The context ID of the chat. Adds an extra layer to the\n            local memory.\"",\n            value=\"\",\n            advanced=True,\n        ),\n        MessageTextInput(\n            name=\"data_template\",\n            display_name=\"Data Template\",\n            value=\"{text}\",\n            advanced=True,\n            info=\"Template to convert Data to Text. If left empty, it will be dynamically\n            set to the Data's text key.\"",\n        ),\n        BoolInput(\n            name=\"clean_data\",\n            display_name=\"Basic Clean Data\",\n            value=True,\n            advanced=True,\n            info=\"Whether to clean data before converting to string.\"",\n        ),\n    ]\n\n    outputs = [\n        Output(\n            display_name=\"Output Message\",\n            name=\"message\",\n            method=\"message_response\",\n        ),\n    ]\n\n    def _build_source(self, id_: str | None,\n        display_name: str | None, source: str | None) -> Source:\n        source_dict = {}\n        if id_:\n            source_dict[\"id\"] = id_\n        if display_name:\n            source_dict[\"display_name\"] =\n            display_name\n        if source:\n            # Handle case where source is a ChatOpenAI object\n            if hasattr(source, \"model_name\"):\n                source_dict[\"source\"] = source.model_name\n            elif hasattr(source, \"model\"):\n                source_dict[\"source\"] = str(source.model)\n            else:\n                source_dict[\"source\"] = str(source)\n        return Source(**source_dict)\n\n    async def\n    message_response(self) -> Message:\n        # First convert the input to string if needed\n        text =\n        self.convert_to_string()\n        # Get source properties\n        source, _, display_name, source_id =\n        self.get_properties_from_source_component()\n        # Create or use existing Message object\n        if isinstance(self.input_value, Message) and not self.is_connected_to_chat_input():\n            message = self.input_value\n            # Update message properties\n            message.text = text\n            # Preserve existing session_id from the incoming message if it exists\n            existing_session_id =\n            message.session_id\n            else:\n                message = Message(text=text)\n            existing_session_id = None\n            # Set message properties\n            message.sender =\n            self.sender\n            message.sender_name = self.sender_name\n            # Preserve session_id from

```

```

incoming message, or use component/graph session_id\n      message.session_id = (\n
self.session_id or existing_session_id or (self.graph.session_id if hasattr(self, \"graph\") else None)
or \"\"\n      )\n      message.context_id = self.context_id\n      message.flow_id =
self.graph.flow_id if hasattr(self, \"graph\") else None\n      message.properties.source =
self._build_source(source_id, display_name, source)\n\n      # Store message if needed\n      if
message.session_id and self.should_store_message:\n          stored_message = await
self.send_message(message)\n          self.message.value = stored_message\n          message =
stored_message\n\n      # Set accumulated token usage from all upstream LLM vertices.\n      #
This must happen AFTER send_message() because streaming captures\n      # usage from chunks
and would overwrite accumulated totals.\n      if hasattr(self, \"_vertex\") and self._vertex is not
None:\n          accumulated_usage = self._vertex._accumulate_upstream_token_usage() # noqa:
SLF001\n          if accumulated_usage:\n              message.properties.usage =
accumulated_usage\n              if self.should_store_message and message.get_id():\n
message = await self._update_stored_message(message)\n              await
self._send_message_event(message, id_=message.get_id())\n\n          self.status = message\n
return message\n\n      def _serialize_data(self, data: Data) -> str:\n          \"\"\n          \"Serialize Data object to
JSON string.\"\n          # Convert data.data to JSON-serializable format\n          serializable_data =
jsonable_encoder(data.data)\n          # Serialize with orjson, enabling pretty printing with indentation\n
json_bytes = orjson.dumps(serializable_data, option=orjson.OPT_INDENT_2)\n          # Convert bytes
to string and wrap in Markdown code blocks\n          return \"```\njson\n\n\" + json_bytes.decode(\"utf-8\")
+ \"\n\n```\n\n      def _validate_input(self) -> None:\n          \"\"\n          \"Validate the input data and raise
ValueError if invalid.\"\n          if self.input_value is None:\n              msg = \"Input data cannot be
None\"\n              raise ValueError(msg)\n          if isinstance(self.input_value, list) and not all(\n
isinstance(item, Message | Data | DataFrame | str) for item in self.input_value\n          ):\n
invalid_types = [\n              type(item).__name__\n          ]\n          for item in self.input_value\n
if not isinstance(item, Message | Data | DataFrame | str)\n          ]\n          msg = f\"Expected Data
or DataFrame or Message or str, got {invalid_types}\"\n          raise TypeError(msg)\n          if not
isinstance(\n              self.input_value,\n              Message | Data | DataFrame | str | list | Generator |
type(None),\n          ):\n              type_name = type(self.input_value).__name__\n              msg =
f\"Expected Data or DataFrame or Message or str, Generator or None, got {type_name}\"\n              raise
TypeError(msg)\n\n          def convert_to_string(self) -> str | Generator[Any, None, None]:\n
          \"\"\n          \"Convert input data to string with proper error handling.\"\n          self._validate_input()\n          if
isinstance(self.input_value, list):\n              clean_data: bool = getattr(self, \"clean_data\", False)\n
return \"\n\n\".join([safe_convert(item, clean_data=clean_data) for item in self.input_value])\n          if
isinstance(self.input_value, Generator):\n              return self.input_value\n          return
safe_convert(self.input_value)\n\n      },\n      \"context_id\": {\n          \"_input_type\": \"MessageTextInput\",\n          \"advanced\": true,\n          \"display_name\": \"Context ID\",\n          \"dynamic\": false,\n          \"info\": \"The context ID of the chat. Adds an extra layer to the local memory.\",\n          \"input_types\": [\n              \"Message\"\n          ],\n      },

```

```

    "list": false,
    "list_add_label": "Add More",
    "load_from_db": false,
    "name": "context_id",
    "override_skip": false,
    "placeholder": "",
    "required": false,
    "show": true,
    "title_case": false,
    "tool_mode": false,
    "trace_as_input": true,
    "trace_as_metadata": true,
    "track_in_telemetry": false,
    "type": "str",
    "value": ""
  },
  "data_template": {
    "_input_type": "MessageTextInput",
    "advanced": true,
    "display_name": "Data Template",
    "dynamic": false,
    "info": "Template to convert Data to Text. If left empty, it will be dynamically set to the
Data's text key.",
    "input_types": [
      "Message"
    ],
    "list": false,
    "list_add_label": "Add More",
    "load_from_db": false,
    "name": "data_template",
    "override_skip": false,
    "placeholder": "",
    "required": false,
    "show": true,
    "title_case": false,
    "tool_mode": false,
    "trace_as_input": true,
    "trace_as_metadata": true,
    "track_in_telemetry": false,
    "type": "str",
    "value": "{text}"
  },
  "input_value": {
    "_input_type": "HandleInput",
    "advanced": false,
    "display_name": "Inputs",

```

```

    "dynamic": false,
    "info": "Message to be passed as output.",
    "input_types": [
        "Data",
        "JSON",
        "DataFrame",
        "Table",
        "Message"
    ],
    "list": false,
    "list_add_label": "Add More",
    "name": "input_value",
    "override_skip": false,
    "placeholder": "",
    "required": true,
    "show": true,
    "title_case": false,
    "trace_as_metadata": true,
    "track_in_telemetry": false,
    "type": "other",
    "value": ""
},
"sender": {
    "_input_type": "DropdownInput",
    "advanced": true,
    "combobox": false,
    "dialog_inputs": {},
    "display_name": "Sender Type",
    "dynamic": false,
    "external_options": {},
    "info": "Type of sender.",
    "name": "sender",
    "options": [
        "Machine",
        "User"
    ],
    "options_metadata": [],
    "override_skip": false,
    "placeholder": "",
    "required": false,
    "show": true,
    "title_case": false,
    "toggle": false,
    "tool_mode": false,
    "trace_as_metadata": true,
    "track_in_telemetry": true,

```

```

    "type": "str",
    "value": "Machine"
  },
  "sender_name": {
    "_input_type": "MessageTextInput",
    "advanced": true,
    "display_name": "Sender Name",
    "dynamic": false,
    "info": "Name of the sender.",
    "input_types": [
      "Message"
    ],
    "list": false,
    "list_add_label": "Add More",
    "load_from_db": false,
    "name": "sender_name",
    "override_skip": false,
    "placeholder": "",
    "required": false,
    "show": true,
    "title_case": false,
    "tool_mode": false,
    "trace_as_input": true,
    "trace_as_metadata": true,
    "track_in_telemetry": false,
    "type": "str",
    "value": "AI"
  },
  "session_id": {
    "_input_type": "MessageTextInput",
    "advanced": true,
    "display_name": "Session ID",
    "dynamic": false,
    "info": "The session ID of the chat. If empty, the current session ID parameter will be
used.",
    "input_types": [
      "Message"
    ],
    "list": false,
    "list_add_label": "Add More",
    "load_from_db": false,
    "name": "session_id",
    "override_skip": false,
    "placeholder": "",
    "required": false,
    "show": true,

```

```

    "title_case": false,
    "tool_mode": false,
    "trace_as_input": true,
    "trace_as_metadata": true,
    "track_in_telemetry": false,
    "type": "str",
    "value": ""
  },
  "should_store_message": {
    "_input_type": "BoolInput",
    "advanced": true,
    "display_name": "Store Messages",
    "dynamic": false,
    "info": "Store the message in the history.",
    "list": false,
    "list_add_label": "Add More",
    "name": "should_store_message",
    "override_skip": false,
    "placeholder": "",
    "required": false,
    "show": true,
    "title_case": false,
    "tool_mode": false,
    "trace_as_metadata": true,
    "track_in_telemetry": true,
    "type": "bool",
    "value": true
  }
},
"tool_mode": false
},
"showNode": false,
"type": "ChatOutput"
},
"dragging": false,
"id": "ChatOutput-D1HOI",
"measured": {
  "height": 52,
  "width": 192
},
"position": {
  "x": 1661.799438406991,
  "y": 230.09731962265113
},
"selected": false,
"type": "genericNode"

```



```
    }  
  ],  
  "viewport": {  
    "x": 136.651004958891,  
    "y": 44.34285273498193,  
    "zoom": 0.5977905174570449  
  }  
},  
"description": "Text Generation Meets Business Transformation.",  
"endpoint_name": null,  
"id": "efb6bca2-9013-4f40-928f-48711256f151",  
"is_component": false,  
"last_tested_version": "1.9.1",  
"locked": false,  
"name": "Event generator POC",  
"tags": []  
}
```